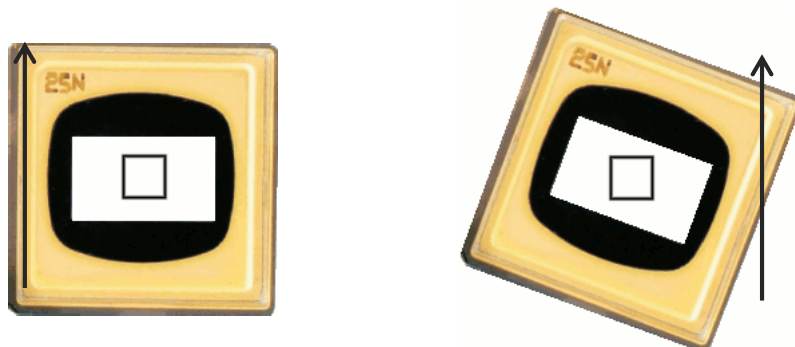


ALP Application Note: On-the-fly rotation of image data using Shear Scrolling

The display of scrolling DMD patterns is commonly required in industrial use cases where a substrate is processed by moving DLP exposure heads (or moving substrates). The well-synchronized scrolling motion of the DMD display keeps the patterns fixed relative to the substrate.

Scrolling patterns from the on-board memory of V-Modules yields significant advantages with respect to the bandwidth requirements for the data upload from PC. Only the data of N new lines are required to generate the next DMD frame, shifted by N lines.

There are advanced exposure approaches in industry, i.e. for resolution enhancement, where the DMD lines are positioned non-perpendicular to the direction of movement. Correspondingly, the pattern scrolling of lines cannot be performed by simple line shifts only, but the patterns read from a data stripe in memory have to be rotated before loading them into the DMD.



Vertical scrolling.

Rotate DMD to scroll direction.

ALP-4.3 and ALP-4.4 provides a mode of operation that enables SuperSpeed V-Modules for being used in such advanced DLP exposure applications, too. This “Shear Scrolling” ALP API extension adds a scrolling display mode for such type of non-linear exposure paths. Long stripes of patterns can be scrolled over the DMD in a direction non-perpendicular to the DMD lines.

ALP-4.3 and ALP-4.4 Shear Scrolling is designed rather universal, supporting even curved paths by line dependent shear values. However, the pattern rotation by straight shear is the most popular use case so far.

This application note provides the mathematics that runs in ALP on-the-flight and the preprocessing required in the computer (PC) that results in a non-distorted, gap-less filled display.

The approach is demonstrated in the “ALP Shearing Sample” application, provided with the ALP Installation.

1 Rotate pictures in two steps

This section describes how the ALP-4.3 and ALP-4.4 API Shear Scrolling extension can be used to rotate patterns during scrolling a pattern sequence from on-board memory at full V-Module speed.

In digital graphics, a rotation with an angle α is described as the following linear transformation of pixel coordinates:

$$\begin{pmatrix} x'' \\ y'' \end{pmatrix} = \begin{bmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{bmatrix} \begin{pmatrix} x \\ y \end{pmatrix} \quad (1)$$

with source pixel (x, y) and DMD pixel (x'', y'') .

A two-step approach is adopted and the transformation (2) is split into two consecutive operations:

- a combined shearing and scaling operation (executed in the PC, before the stripe of patterns is uploaded to the V-Module by *AlpSeqPut*) and
- a shearing operation executed in the V-Module hardware by ALP during the pattern sequence display

$$\begin{pmatrix} x'' \\ y'' \end{pmatrix} = \underbrace{\begin{bmatrix} 1 & -\tan \alpha \\ 0 & 1 \end{bmatrix}}_{V-Module: ALP_X_SHEAR} \underbrace{\begin{bmatrix} 1/\cos \alpha & 0 \\ \sin \alpha & \cos \alpha \end{bmatrix}}_{PC} \begin{pmatrix} x \\ y \end{pmatrix} \quad (2)$$

The substantial benefit of such two-step approach is the feasibility of on-the-fly shearing in the V-Module hardware without any loss of performance in display, i.e. the specified frame rates of SuperSpeed V-Modules are maintained.

1.1 Horizontal Shearing

The ALP Shearing Sample uses ALP_X_SHEAR to transform coordinates from sequence pixel (x', y') to DMD pixel (x'', y'') by the formula below:

$$\begin{pmatrix} x'' \\ y'' \end{pmatrix} = \underbrace{\begin{bmatrix} 1 & -\tan \alpha \\ 0 & 1 \end{bmatrix}}_{ALP_X_SHEAR} \begin{pmatrix} x' \\ y' \end{pmatrix} \quad (3)$$

The source code generating the according *tAlpShearTable* can be found in file “ALP Shearing Sample.cpp”.

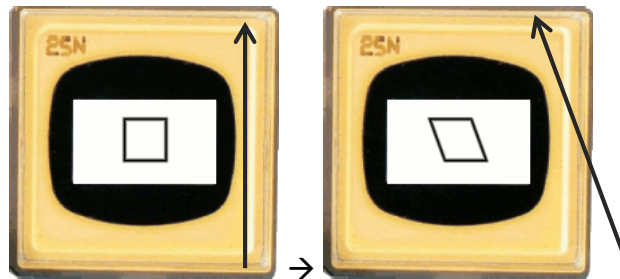
```
// set up shear-table 1 (linear shearing)
ShearTableLinear.nOffset = 0;
ShearTableLinear.nSize = nRows;

for(long i=0; i < nRows; i++)
    ShearTableLinear.nShiftDistance[i] = (long) (fTanAlpha*i);

/* ... */

AlpProjControlEx(AlpDevId, ALP_X_SHEAR, &ShearTableLinear);
AlpSeqControl(AlpDevId, AlpSeqId1, ALP_X_SHEAR_SELECT, ALP_ENABLE);
```

The result is shown in the pictures below. The sequence data must obviously be adjusted.



1.2 Preprocessing

The computation on the PC side has the form:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{bmatrix} 1/\cos \alpha & 0 \\ \sin \alpha & \cos \alpha \end{bmatrix} \begin{pmatrix} x \\ y \end{pmatrix} \quad (4)$$

with source pixel (x, y) and DMD pattern pixel (x', y') (i.e. a pixel of the data that is uploaded by *AlpSeqPut*).

Formula (4) delivers new pattern pixel coordinates for each source pixel. That could cause some pixels to be written twice, or not being written at all (gaps).

Therefore, we recommend an implementation which computes the pixels from the display pixels “point of view”:

$$\begin{pmatrix} x \\ y \end{pmatrix} = \begin{bmatrix} \cos \alpha & 0 \\ -\sin \alpha & 1/\cos \alpha \end{bmatrix} \begin{pmatrix} x' \\ y' \end{pmatrix} \quad (5)$$

Formula (5) is implemented in the “ALP Shearing Sample”. The sample code is provided with the ALP-4.3 and ALP-4.4 API Installation (see “ALP Shearing Sample.cpp” for details).

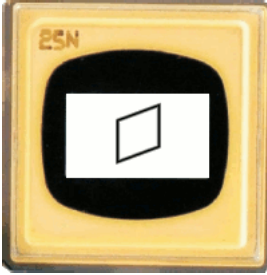
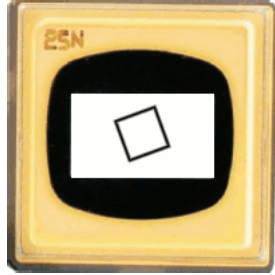
```

for(long y=0; y < nMaxShearDataRow; y++)
{
    for(long x=0; x < nColumns; x++)
    {
        double const fSrcX = x*fCosAlpha,
                     fSrcY = y/fCosAlpha - x*fSinAlpha;
        long const nSrcX = (long)(fSrcX),
                  nSrcY = (long)(fSrcY);

        if (nSrcY<0 || nSrcY>=nFrames*nRows)
            // area vertically outside pSeqData1
            pSeqData2[y*nColumns + x] = 0;
        else
            pSeqData2[y*nColumns + x] = pSeqData1[nSrcY*nColumns + nSrcX];
    }
}

```

The steps of transformation with the intermediate and final results are schematically illustrated in the pictures below.

 Source data	 Preprocessing	 AlpSeqPut	 ALP_X_SHEAR_SELECT = ENABLE
---	---	--	---